

DOCMitra AI

Complete Technology Breakdown — Features, Technologies, Architecture & Viva Notes

1. Project Overview

DOCMitra AI / DeptSphere AI is a college department-management system with separate Faculty and Admin functionality. Faculty manage department documents, Student Forum AI, reminders and profiles. Admin manages faculty, monitors documents, receives upload notifications and uses administrative features.

2. React.js

Used to build the complete frontend UI and reusable components such as Login, Dashboard, Repository, Upload, Student Forum AI, Profile, Faculty Management, Admin Dashboard and Admin Documents.

3. Vite

Used as the React development and production build tool. It provides fast development, hot reload and optimized builds.

4. React Router DOM

Used for client-side navigation between routes such as /dashboard, /repository, /profile, /student-forum-ai, /admin-dashboard, /faculty-management and /admin-documents.

5. Axios

Used for frontend-backend REST API communication. The shared API instance uses `http://localhost:5000/api` and automatically attaches the JWT Authorization header.

6. Node.js

Runs the server-side JavaScript. It handles authentication, APIs, uploads, repository operations, notifications, reminders, AI requests and database communication.

7. Express.js

Backend framework used to build REST APIs and organize routes, controllers and middleware.

8. MongoDB

NoSQL database used to store Faculty accounts, Repository metadata, reminders, notifications and other application data.

9. Mongoose

ODM used to connect Node.js with MongoDB and define schemas/models and document relationships.

10. JWT Authentication

JSON Web Token is used for authentication. After login, the backend issues a token containing the user ID and role. The token is sent with protected API requests.

11. Role-Based Access Control

The system has Faculty and Admin roles. Faculty access Dashboard, Repository, Student Forum AI, Smart Reminders and Profile. Admin access Admin Dashboard, Faculty Management, Admin Documents, notifications and planned Analytics.

12. ProtectedRoute

Restricts protected pages to authenticated users and can enforce a specific role such as admin.

13. CAPTCHA

Adds a security layer to authentication to help distinguish human users from automated bots and reduce automated login attempts.

14. bcrypt

Used for password hashing and verification so passwords are not stored as plain text.

15. Forgot Password

Provides an account-recovery flow through the Forgot Password page and backend reset process.

16. Multer

Handles multipart/form-data uploads for Repository documents and profile images. Supported Repository types include PDF, DOC, DOCX, PPT, PPTX, TXT, JPG, JPEG and PNG, with a 20 MB upload limit.

17. File Storage

Currently uploaded files are stored locally in the server uploads/ directory. MongoDB stores related metadata/path. AWS S3 is planned, not currently active.

18. Repository

Central document-management module supporting upload, view, search, filtering, edit and delete. Metadata includes title, description, subject, department, session, term, category, file and uploader.

19. Session and Term

Academic organization uses Session 2026-2027 with Even and Odd terms rather than semester-number navigation.

20. Current Search

Repository search is currently text-based and primarily searches repository information such as document titles. It is not yet semantic/vector search.

21. Semantic Search — Planned

Future flow: User Query → Embedding → Vector Database → Similarity Search → Relevant Documents. LangChain, ChromaDB/vector databases and embeddings are planned but not fully implemented.

22. Google Gemini

Provides AI functionality such as Dashboard AI and Student Forum AI. AI requests are handled through the backend.

23. Student Forum AI

Specialized workflow for D.50 Student Forum: Create → Add/Upload Image → AI Generation → Preview → PDF → Save to Repository. Final output is PDF rather than DOCX.

24. ReactMarkdown + rehypeRaw

ReactMarkdown renders AI-generated Markdown. rehypeRaw supports permitted raw HTML inside Markdown where required.

25. html2canvas + jsPDF

html2canvas captures the rendered document content and jsPDF creates the final PDF. Flow: HTML preview → canvas → PDF.

26. Smart Reminders

Faculty-only feature containing title, description, type, priority, due date, status, read/unread state and creator. Faculty can create, read, complete and delete reminders.

27. Admin Notifications

Separate notification system for Admin. Faculty document uploads can create notifications showing faculty name, document name, session and term.

28. Notification API

Main endpoints are GET /api/notifications and PUT /api/notifications/:id/read. Notifications contain title, message, type, documentId, recipientRole, isRead and timestamps.

29. Notification Flow

Faculty uploads document → Repository saved → Faculty identified → Admin notification created → Admin Header fetches it → unread badge/message displayed.

30. Student Forum Notification Gap

Normal Repository uploads use the notification-enabled upload controller. Student Forum AI saves through /student-forum-ai/save, so that save path still needs equivalent Admin notification behavior if required.

31. Profile Photo

Faculty profile photos are uploaded with Multer, served by the backend and displayed in the Header and Profile page.

32. AppLayout

Reusable page shell containing Header, Sidebar and page content, avoiding duplicated layout code.

33. Custom CSS

Uses a dark navy sidebar, white cards, light-gray background, purple primary accent (#6C51F2), rounded corners, shadows, active navigation and responsive layouts.

34. Calendar

Dashboard Calendar provides date/event visibility. It is currently a UI component rather than a full external calendar-service integration.

35. Admin Dashboard

Contains Total Faculty, Total Documents, Active Faculty, Recent Documents, Calendar, Admin Actions and AI Assistant. Smart Reminders are intentionally excluded.

36. Faculty Management

Admin-side module for administrative control of faculty accounts.

37. Admin Documents

Admin document oversight including open, edit and delete. It uses the same Repository data and should be wrapped in the shared AppLayout plus admin ProtectedRoute.

38. Analytics — Planned

Admin Analytics is planned to calculate Total Documents, Total Faculty, Active Faculty, Even/Odd distribution, categories, upload activity and faculty contribution from MongoDB.

39. Admin AI Assistant

General flow: User question → React → Express → Gemini → AI response → React.

40. Implemented vs Planned

Implemented/current: React, Vite, Node.js, Express, MongoDB, Mongoose, JWT, bcrypt, CAPTCHA, Multer, local storage, Gemini, Repository, Student Forum AI, PDF generation, Smart Reminders, Profile, Admin Dashboard, Faculty Management, Admin Documents and in-app notifications. Planned/not fully implemented: LangChain, ChromaDB/vector database, semantic search, AWS S3, full Analytics and SMS notifications.

41. Complete Technology Stack

Area	Technology	Purpose
Frontend	React.js	Build user interface
Build	Vite	Development/production build
Routing	React Router DOM	Client-side navigation
API	Axios	Frontend-backend communication
Backend	Node.js	Server-side JavaScript
Backend	Express.js	REST APIs
Database	MongoDB	Application data
ODM	Mongoose	MongoDB schemas/models
Authentication	JWT	Authentication/authorization
Security	bcrypt	Password hashing
Security	CAPTCHA	Bot protection
Uploads	Multer	File uploads
AI	Google Gemini	AI assistant/content generation
Markdown	ReactMarkdown	Render AI Markdown
Markdown	rehypeRaw	Raw HTML support
PDF	html2canvas	Capture rendered content
PDF	jsPDF	Generate PDFs
Storage	Local uploads	Current file storage
Access	ProtectedRoute + roles	Restrict access
Planned Search	Embeddings + Vector DB	Semantic retrieval
Planned	LangChain / ChromaDB	Future semantic search
Planned Cloud	AWS S3	Future cloud storage

42. One-Line Viva Answers

Technology	Viva answer
React	Builds the frontend UI.
Vite	Fast React development/build tool.
React Router	Handles page navigation.
Axios	Connects frontend and backend APIs.
Node.js	Runs the backend JavaScript environment.
Express.js	Builds REST APIs.
MongoDB	Stores application data.
Mongoose	Defines MongoDB schemas/models.
JWT	Provides authentication tokens.
bcrypt	Hashes and verifies passwords.
CAPTCHA	Reduces automated/bot login attempts.
Multer	Processes file uploads.
Gemini	Provides AI assistance and content generation.
ReactMarkdown	Renders AI-generated Markdown.
html2canvas	Captures rendered document content.

Technology	Viva answer
jsPDF	Generates PDF documents.
ProtectedRoute	Restricts protected pages.
Smart Reminders	Helps Faculty manage reminders/tasks.
Notifications	Lets Admin receive document-upload events.

43. Full System Flow

User → React/Vite → Axios → Express/Node.js → JWT/Role Validation → Controller/Service → MongoDB via Mongoose.

Document upload → Multer → local uploads/ storage + Repository metadata in MongoDB.

Faculty upload → Repository saved → Admin Notification created → Admin Header displays notification.

AI request → React → Express → Gemini → AI response → React.

Student Forum AI → Create → Image → Gemini-assisted content → Preview → html2canvas/jsPDF → PDF → Save to Repository.

Future semantic search → User query → Embedding → Vector Database → Similarity Search → Relevant documents.

44. Presentation/Viva Summary

Suggested answer: DOCMitra AI is a React and Vite-based frontend connected to a Node.js and Express backend through REST APIs using Axios. MongoDB with Mongoose stores application data. JWT provides authentication and role-based access control. bcrypt protects passwords and CAPTCHA adds protection against automated login attempts. Multer handles uploads. Gemini provides AI functionality, while html2canvas and jsPDF generate Student Forum PDFs. Faculty manage Repository documents and Smart Reminders; Admin manages faculty, documents and upload notifications. Semantic search, vector databases, AWS S3 and full Analytics are planned extensions rather than fully implemented current features.